Manual de CI/CD com GitHub e Coolify

BiblioDist — Sistema de Gerenciamento de Biblioteca Distribuída

Disciplina: Sistemas Operacionais II
Professor: Douglas
Instituição: FEMA — Fundação Educacional do Município de Assis
Aluno: Arthur Naoto Miura
RA: 2311600029
Data: Novembro de 2026

> ***Agradecimento:*** *Este trabalho contou com o auxílio do aluno* ***Uryel****, que colaborou colocando os arquivos no diretório correto do servidor durante a etapa de deploy no Coolify.*

Sumário

1. Visão Geral do Fluxo CI/CD
2. Criação do Repositório no GitHub
3. Estrutura do Projeto no GitHub
4. Criação do Projeto no Coolify
5. Integração GitHub Actions com Coolify
6. Fluxo Completo de Deploy Automático
7. Acesso à Aplicação pela URL
8. Resultado Final

---

1. Visão Geral do Fluxo CI/CD

O fluxo de **Integração Contínua / Entrega Contínua (CI/CD)** do BiblioDist funciona da seguinte forma:





Resumo: a cada `git push` para o branch `master`, o GitHub Actions dispara automaticamente um webhook que instrui o Coolify a fazer o rebuild e redeploy de todos os containers da aplicação.

---

2. Criação do Repositório no GitHub

Passo 1 — Criar o repositório

Acesse `github.com`, clique em **New repository** e configure:

Campo	Valor
Repository name	`bibliodist`
Visibility	Public
Initialize with README	Não (já temos o código local)

Clique em **Create repository**.

Passo 2 — Conectar o repositório local

No terminal, dentro da pasta do projeto:

```
git init
git add .
git commit -m "feat: commit inicial"
git branch -M master
git remote add origin https://github.com/R2nin/bibliodist.git
git push -u origin master
```

Passo 3 — Verificar o repositório

Após o push, o repositório estará acessível em:
`https://github.com/R2nin/bibliodist`

O branch principal é o `master`. Todo push para esse branch aciona o pipeline de deploy.

---

3. Estrutura do Projeto no GitHub

O repositório é um **monorepo** com um serviço por pasta. Cada serviço tem seu próprio `Dockerfile`:

```
bibliodist/
├── .github/
│   └── workflows/
│       └── deploy.yml          ← pipeline CI/CD
│
├── api-web/                    ← Django + Gunicorn
│   ├── Dockerfile
│   ├── requirements.txt
│   └── ...
│
├── grpc-service/               ← servidor gRPC
│   ├── Dockerfile
│   ├── requirements.txt
│   └── ...
│
├── worker/                     ← consumidor RabbitMQ
│   ├── Dockerfile
│   ├── requirements.txt
│   └── ...
│
├── proto/                      ← contrato gRPC (.proto)
│
├── docker-compose.yml          ← uso local/dev
├── docker-compose.coolify.yml  ← uso em produção (Coolify)
└── README.md
```

O arquivo **`docker-compose.coolify.yml`** é o que o Coolify usa. Nele, o `context` de cada serviço aponta diretamente para o repositório no GitHub:

```
web:
  build:
    context: https://github.com/R2nin/bibliodist.git#master
    dockerfile: api-web/Dockerfile
```

Isso significa que, ao fazer deploy, o Coolify clona o branch `master` diretamente do GitHub e compila as imagens no servidor.

---

4. Criação do Projeto no Coolify

O que é o Coolify?

Coolify é uma plataforma PaaS (Platform as a Service) *self-hosted*, similar ao Heroku ou Railway, mas instalada em servidor próprio. Permite fazer deploy de aplicações via Docker Compose com painel gráfico, certificados SSL automáticos e domínios personalizados.

A instância utilizada está instalada no servidor do professor em `computacao.fema.edu.br`.

Passo 1 — Acessar o painel Coolify

Acesse o painel em `http://computacao.fema.edu.br:8000` (ou porta configurada) e faça login com as credenciais fornecidas pelo administrador.

Passo 2 — Criar um novo projeto

No menu lateral, clique em **Projects** → **+ New Project**.

Preencha:

- **Name:** `BiblioDist`
- **Description:** Sistema de Gerenciamento de Biblioteca Distribuída

Clique em **Create Project**.

Passo 3 — Criar um novo ambiente

Dentro do projeto, clique em **+ New Environment**:

- **Name:** `production`

Passo 4 — Adicionar um novo recurso (Docker Compose)

Dentro do ambiente `production`, clique em **+ New Resource** → **Docker Compose**.

Selecione a origem:

- **Raw Docker Compose** (colar o conteúdo diretamente)

Passo 5 — Colar o `docker-compose.coolify.yml`

Cole o conteúdo do arquivo `docker-compose.coolify.yml` do repositório. Esse arquivo define os 6 serviços:

```
services:
  db:        # PostgreSQL 16
  rabbitmq:  # RabbitMQ 3.13
  migrate:   # Init container — roda migrate e sai
  web:       # Django + Gunicorn (porta 8000)
  grpc:      # Servidor gRPC (porta 50051)
  worker:    # Consumidor RabbitMQ
```

Cada serviço com `build.context` apontando para o GitHub é compilado pelo Coolify no momento do deploy.

Passo 6 — Configurar variáveis de ambiente

No painel do serviço `web`, adicione as variáveis de ambiente:

Variável	Valor
`DATABASE_URL`	`postgresql://bibliodist:bibliodist@db:5432/bibliodist`
`RABBITMQ_URL`	`amqp://guest:guest@rabbitmq:5672/`
`GRPC_HOST`	`grpc`
`GRPC_PORT`	`50051`
`DEBUG`	`False`
`ALLOWED_HOSTS`	`bibliodist.187.33.201.107.sslip.io,localhost`
`CSRF_TRUSTED_ORIGINS`	`https://bibliodist.187.33.201.107.sslip.io`

Passo 7 — Configurar o domínio

No serviço `web`, em **Domains**, adicione:

```
https://bibliodist.187.33.201.107.sslip.io
```

O domínio `sslip.io` é um serviço DNS público que resolve qualquer subdomínio no formato `IP.sslip.io` automaticamente para o IP correspondente. Isso elimina a necessidade de configurar registros DNS manualmente.

O Coolify gera o certificado SSL via **Let's Encrypt** automaticamente para esse domínio.

Passo 8 — Obter o Webhook de Deploy

No Coolify, dentro do projeto, acesse **Settings** → **Webhooks** (ou na barra lateral do recurso).

Copie a **Deploy Webhook URL**. Ela terá o formato:

```
https://computacao.fema.edu.br/api/v1/deploy?uuid=XXXXXXXX&token=YYYYYYYY
```

Essa URL será usada no próximo passo para integrar com o GitHub Actions.

Passo 9 — Primeiro deploy manual

Clique em **Deploy** para fazer o primeiro deploy. O Coolify vai:

1. Clonar o repositório do GitHub
2. Compilar as imagens Docker de cada serviço
3. Inicializar os containers na ordem correta (healthchecks garantem isso)
4. Rotear o tráfego via Traefik (proxy reverso interno)

Aguarde todos os containers ficarem com status **Running** (o `migrate` fica **Exited 0** — normal, é um init container).

---

5. Integração GitHub Actions com Coolify

Como funciona

O **GitHub Actions** é a plataforma de CI/CD nativa do GitHub. Um arquivo YAML em `.github/workflows/` define quando e como o pipeline é executado.

A integração com o Coolify funciona por **webhook**: o GitHub Actions envia uma requisição HTTP para a URL de webhook do Coolify, que por sua vez inicia o processo de rebuild e redeploy.

Passo 1 — Adicionar o Webhook como Secret no GitHub

No repositório GitHub, acesse:
Settings → **Secrets and variables** → **Actions** → **New repository secret**

Configure:

- **Name:** `COOLIFY_WEBHOOK_URL`
- **Secret:** cole a URL copiada do Coolify no passo anterior

Clique em **Add secret**.

> *O Secret fica criptografado e nunca aparece nos logs do Actions — apenas o próprio workflow pode utilizá-lo.*

Passo 2 — O arquivo de workflow

O arquivo `.github/workflows/deploy.yml` já está no repositório:

```yaml
name: Deploy para Coolify

on:
  push:
    branches:
      - master

jobs:
  deploy:
    name: Acionar deploy no Coolify
    runs-on: ubuntu-latest

    steps:
      - name: Notificar Coolify via webhook
        run: |
          curl --silent --fail --show-error \
            --request GET \
            "${{ secrets.COOLIFY_WEBHOOK_URL }}"
        env:
          COOLIFY_WEBHOOK_URL: ${{ secrets.COOLIFY_WEBHOOK_URL }}
```

Explicação linha a linha:

Elemento	Significado
`on: push: branches: master`	Aciona o pipeline a cada push no branch `master`

Elemento	Significado
`runs-on: ubuntu-latest`	Executa em um runner Linux gratuito do GitHub
`curl --silent --fail`	Chama a URL do webhook; falha com erro se o HTTP não for 2xx
`secrets.COOLIFY_WEBHOOK_URL`	Referência ao secret criado no passo anterior

Passo 3 — Verificar a execução no GitHub

Após qualquer `git push` para `master`, acesse:
GitHub → **repositório** → aba **Actions**

Você verá o workflow em execução (ícone amarelo) ou concluído (ícone verde ✓).

Clique no workflow para ver os logs de cada step. O step **Notificar Coolify via webhook** deve exibir resposta HTTP 200.

---

6. Fluxo Completo de Deploy Automático

Após toda a configuração, o fluxo se torna completamente automático:

Ciclo de desenvolvimento

```
1. Desenvolvedor edita o código localmente
        |
        ▼
2. git add . && git commit -m "feat: nova funcionalidade"
        |
        ▼
3. git push origin master
        |
        ▼
4. GitHub detecta o push → dispara o workflow deploy.yml
        |
        ▼
5. GitHub Actions executa: curl COOLIFY_WEBHOOK_URL
        |
        ▼
6. Coolify recebe o webhook → inicia rebuild
   - Clona o repositório (branch master)
   - Compila imagens: web, grpc, worker
   - Para os containers antigos
   - Sobe os novos containers
   - Aplica migrations (init container migrate)
        |
        ▼
```

```
7. Nova versão disponível em ~2-3 minutos
   https://bibliodist.187.33.201.107.sslip.io
```

Exemplo real de deploy

Commit `c9bef1b` — *fix: remover SCRIPT_NAME do environ no wsgi.py*:

1. Push realizado às 14:32
2. GitHub Actions iniciou às 14:32 (latência < 10 s)
3. Webhook enviado ao Coolify às 14:32
4. Rebuild concluído às 14:34 (~2 min de build)
5. Correção disponível em produção às 14:34

---

7. Acesso à Aplicação pela URL

Como o roteamento funciona no Coolify

O Coolify usa o **Traefik** como proxy reverso. O Traefik:

1. Escuta nas portas 80 (HTTP) e 443 (HTTPS)
2. Lê os labels Docker de cada container para saber qual domínio rotear
3. Obtém e renova certificados SSL automaticamente via Let's Encrypt

```
Internet
   │
   │ HTTPS :443
   ▼
┌──────────────────────────────────────┐
│   Traefik (proxy reverso)            │
│   bibliodist.187.33.201.107.sslip.io │
│        │                             │
│        └──► container web :8000      │
└──────────────────────────────────────┘
```

O serviço `web` **não** expõe a porta 8000 diretamente na internet — apenas o Traefik tem acesso a ela internamente na rede Docker.

O domínio sslip.io

`sslip.io` é um serviço de DNS público que resolve automaticamente subdomínios no formato:

```
<qualquer-prefixo>.<IP-com-pontos-substituídas-por-hífens>.sslip.io
          ↓
      IP original
```

Exemplo:

```
bibliodist.187.33.201.107.sslip.io  →  187.33.201.107
```

Isso permite apontar um domínio para qualquer IP sem precisar configurar registros DNS — ideal para ambientes acadêmicos.

URL amigável com encurtador de links

A URL técnica gerada pelo sslip.io (`bibliodist.187.33.201.107.sslip.io`) é longa e difícil de memorizar. Para facilitar o acesso e a divulgação, foi utilizado o serviço de encurtamento de links **TinyURL**, criando uma URL amigável:

	URL
Acesso rápido (TinyURL)	`https://tinyurl.com/femabibv2`
URL técnica completa	`https://bibliodist.187.33.201.107.sslip.io`

O TinyURL redireciona automaticamente (HTTP 301) para a URL original no Coolify, sem qualquer configuração adicional no servidor.

URLs disponíveis em produção

Serviço	URL
Aplicação principal	`https://tinyurl.com/femabibv2`
Login	`https://bibliodist.187.33.201.107.sslip.io/accounts/login/`
Painel Admin	`https://bibliodist.187.33.201.107.sslip.io/admin/`
Revisão PRISMA	`https://bibliodist.187.33.201.107.sslip.io/prisma/`
Lista de Livros	`https://bibliodist.187.33.201.107.sslip.io/livros/`

---

8. Resultado Final

Status dos containers em produção

Container	Status	Função
`db`	Running (healthy)	PostgreSQL 16 — banco de dados
`rabbitmq`	Running (healthy)	RabbitMQ 3.13 — fila de mensagens
`migrate`	Exited (0)	Init container — aplica migrations

Container	Status	Função
`web`	Running	Django + Gunicorn — interface web
`grpc`	Running	Servidor gRPC — verificação de disponibilidade
`worker`	Running	Consumidor RabbitMQ — notificações assíncronas

Resumo da pipeline CI/CD

Etapa	Ferramenta	Trigger
Versionamento do código	Git + GitHub	Manual (git push)
Execução do pipeline	GitHub Actions	Automático (push no master)
Notificação de deploy	curl webhook	Automático (Actions)
Build das imagens	Docker (no servidor)	Automático (Coolify)
Deploy dos containers	Docker Compose	Automático (Coolify)
Roteamento + SSL	Traefik + Let's Encrypt	Automático (Coolify)

Repositório

`https://github.com/R2nin/bibliodist`

Todos os arquivos de configuração do CI/CD estão versionados no repositório:

- `.github/workflows/deploy.yml` — workflow do GitHub Actions
- `docker-compose.coolify.yml` — definição dos serviços para produção

> ***Observação:*** *Este documento reflete o estado do projeto em 11 de novembro de 2026. Até a data limite de entrega (17/11/2026), novas funcionalidades poderão ser incorporadas ao sistema e documentadas em versão atualizada deste manual.*

Manual elaborado como parte da avaliação da disciplina de Sistemas Operacionais II — FEMA, Novembro de 2026.